

Documento de Cambios - Proyecto SGE

Fecha: 11/06/2026

1. Introducción

Este documento detalla todos los cambios realizados al proyecto SGE (Sistema de Gestión de Expedientes) como resultado de las correcciones solicitadas. Se implementaron cuatro mejoras principales enfocadas en la arquitectura, consistencia y limpieza del código.

2. Corrección 1: Ordenamiento de Listas

2.1 Descripción del Problema

Las listas devueltas por los UseCase de consulta no tenían ningún criterio de ordenamiento definido, lo que resultaba en datos desordenados y una experiencia inconsistente para el usuario.

2.2 Solución Implementada

Se agregó ordenamiento por FechaCreacion en ambos UseCase de consulta:

Archivos Modificados:

- **ListarExpedientesUseCase.cs**
 - Se modificó el método Ejecutar para retornar la lista ordenada por FechaCreacion en orden ascendente.
 - Cambio específico: `listaDinamica.OrderBy(e => e.FechaCreacion).ToList()`
- **ListarTramitesPorExpedienteUseCase.cs**
 - Se modificó el método Ejecutar para retornar la lista de trámites ordenada por FechaCreacion.
 - Cambio específico: `dtos.OrderBy(t => t.FechaCreacion).ToList()`

2.3 Justificación

- Proporciona una experiencia de usuario predecible y consistente.
- Los datos históricos se presentan en orden cronológico (más antiguos primero).
- Mejora la usabilidad de la aplicación al permitir al usuario seguir el flujo temporal de los expedientes.

3. Corrección 2: UseCase de Mutación Retornan Response

3.1 Descripción del Problema

Los UseCase que modificaban datos (crear, actualizar, eliminar) tenían inconsistencia en sus patrones: algunos retornaban Response (AgregarExpedienteUseCase) mientras que otros retornaban void. Esto violaba el patrón Request/Response y reducía la consistencia arquitectónica.

3.2 Solución Implementada

Se crearon Response DTOs para todos los UseCase de mutación (aunque algunos estén vacíos), manteniendo consistencia con el patrón Request/Response.

3.2.1 DTOs Creados

DTO Response	UseCase Asociado
BajaExpedienteResponse	BajaExpedienteUseCase
ModificarCaratulaResponse	ModificarCaratulaExpedienteUseCase
CambiarEstadoResponse	CambiarEstadoExpedienteUseCase
ModificarTramiteResponse	ModificarTramiteUseCase
BajaTramiteResponse	BajaTramiteUseCase

3.2.2 Archivos Modificados

- ExpedienteDTOs.cs - Agregados 3 Response DTOs
- TramiteDTOs.cs - Agregados 2 Response DTOs
- BajaExpedienteUseCase.cs - Ahora retorna BajaExpedienteResponse()
- ModificarCaratulaExpedienteUseCase.cs - Ahora retorna ModificarCaratulaResponse()
- CambiarEstadoExpedienteUseCase.cs - Ahora retorna CambiarEstadoResponse()
- ModificarTramiteUseCase.cs - Ahora retorna ModificarTramiteResponse()
- BajaTramiteUseCase.cs - Ahora retorna BajaTramiteResponse()

3.3 Justificación

Consistencia Arquitectónica: Todos los UseCase siguen el mismo patrón (entrada Request, salida Response).

Extensibilidad Futura: Si en el futuro se necesita retornar información adicional (IDs, mensajes, etc.), los Response DTOs pueden ampliarse sin cambiar el contrato del método.

Simetría: Mantiene simetría conceptual: operación que entra con datos (Request) sale con respuesta (Response).

4. Corrección 3: UseCase de Consulta con Request Objects

4.1 Descripción del Problema

Los UseCase de consulta no encapsulaban sus parámetros en Request objects:

- ListarExpedientesUseCase.Ejecutar() - Sin parámetros
- ListarTramitesPorExpedienteUseCase.Ejecutar(Guid expedienteId) - Parámetro desnudo

4.2 Solución Implementada

Se crearon Request objects para encapsular los parámetros y se modificaron los métodos Ejecutar.

4.2.1 Request Objects Creados

- **ListarExpedientesRequest** (vacío por ahora, permite agregar filtros en el futuro)

- **ListarTramitesPorExpedienteRequest**(Guid ExpedienteId) - Encapsula el ID del expediente

4.2.2 Cambios en Métodos

- ListarExpedientesUseCase:
 - ANTES: public IEnumerable<ExpedienteDetalleDTOs> Ejecutar()
 - DESPUES: public IEnumerable<ExpedienteDetalleDTOs> Ejecutar(ListarExpedientesRequest request)
- ListarTramitesPorExpedienteUseCase:
 - ANTES: public IEnumerable<TramiteDetalleDTO> Ejecutar(Guid expedienteId)
 - DESPUES: public IEnumerable<TramiteDetalleDTO> Ejecutar(ListarTramitesPorExpedienteRequest request)

4.2.3 Actualización en Program.cs

- Se actualizaron todas las llamadas a estos UseCase para pasar el Request object:
 - listarExpedientes.Ejecutar(new ListarExpedientesRequest())
 - listarTramites.Ejecutar(new ListarTramitesPorExpedienteRequest(expedienteId))

4.3 Justificación

Abierto/Cerrado: Permite agregar nuevos parámetros (filtros, paginación, ordenamiento) sin cambiar la firma del método.

Mantenibilidad: Los clientes que usan estos UseCase no necesitan ser modificados si se agregan nuevos campos al Request.

Consistencia: Todos los UseCase (consulta y mutación) siguen el mismo patrón Request/Response.

5. Corrección 4: Limpieza de Acentos en Identificadores Técnicos

5.1 Descripción del Problema

Los comentarios técnicos contenían acentos y caracteres especiales del español, lo que no es una buena práctica en código técnico y puede causar problemas de encoding.

5.2 Solución Implementada

Se removieron todos los acentos de comentarios técnicos, manteniendo el código limpio y compatible.

5.2.1 Cambios Realizados

CON ACENTO	SIN ACENTO
Carátula	Caratula
Trámite	Tramite
Trámites	Tramites
Automático	Automatico

Última	Ultima
Última modificación	Ultima modificacion
Método	Metodo
Válido	Valido
Físicamente	Fisicamente
Resolución	Resolucion
Próxima	Proxima

5.2.2 Archivos Afectados

- Expediente.cs - Comentarios sobre Caratula y Tramites
- Caratula.cs - Comentarios sobre validacion
- ContenidoTramite.cs - Comentarios sobre validacion
- Tramite.cs - Comentarios sobre Metodo y fechas
- ExpedienteDTOs.cs - Comentarios sobre DTOs
- BajaTramiteUseCase.cs - Comentarios sobre persistencia
- TramiteTxtRepository.cs - Mensajes de error
- Program.cs - Salida de consola

5.3 Justificación

Estándares de Código: Los comentarios y mensajes técnicos en código deben estar en inglés o español sin acentos.

Compatibilidad: Evita problemas de encoding en diferentes plataformas y editores.

Profesionalismo: Mantiene el código limpio y profesional, siguiendo buenas prácticas internacionales.

6. Errores Encontrados y Corregidos

6.1 Error de Compilación: CS7036

Después de agregar los Request objects a los UseCase de consulta, se generó un error de compilación:

- error CS7036: No se ha dado ningún argumento que corresponda al parámetro requerido "request"

6.1.1 Causa

Program.cs realizaba llamadas a `listarExpedientes.Ejecutar()` sin parámetros, pero después del cambio el método ahora requiere un parámetro `ListarExpedientesRequest`.

6.1.2 Solución

Se actualizaron todas las llamadas en Program.cs (líneas 52, 64, 73, 84 y 97) para pasar el Request object requerido:

- `listarExpedientes.Ejecutar(new ListarExpedientesRequest())`

6.2 Resultado Final

- La compilación se completó exitosamente sin errores después de aplicar estos cambios.

7. Resumen de Archivos Modificados

Archivo	Cambios	Correcciones
ExpedienteDTOs.cs	Agregados 3 Response DTOs + 1 Request DTO	2, 3
TramiteDTOs.cs	Agregados 2 Response DTOs	2
ListarExpedienteUseCase.cs	Modificado método Ejecutar con Request + Ordenamiento	1, 3
ListarTramitesPorExpediente UseCase.cs	Modificado método Ejecutar con Request + Ordenamiento	1, 3
BajaExpedienteUseCase.cs	Modificado para retornar Response	2
ModificarCaratulaExpediente UseCase.cs	Modificado para retornar Response	2
CambiarEstadoExpedienteUseCase.cs	Modificado para retornar Response + Acentos removidos	2, 4
ModificarTramiteUseCase.cs	Modificado para retornar Response + Acentos removidos	2, 4

- BajaTramiteUseCase.cs: Modificado para retornar Response + Acentos removidos (Corrección 2, 4)
- Program.cs: Actualizadas llamadas a UseCase + Acentos removidos (Corrección 1, 3, 4)
- Expediente.cs: Acentos removidos de comentarios (Corrección 4)
- Caratula.cs: Acentos removidos de comentarios (Corrección 4)
- ContenidoTramite.cs: Acentos removidos de comentarios (Corrección 4)
- Tramite.cs: Acentos removidos de comentarios (Corrección 4)
- TramiteTxtRepository.cs: Acentos removidos de comentarios y errores (Corrección 4)

8. Conclusión

Se implementaron exitosamente las cuatro correcciones solicitadas al proyecto SGE:

1. Ordenamiento consistente de listas para mejorar la experiencia del usuario.
2. Patrón Request/Response consistente en todos los UseCase de mutación.
3. Encapsulación de parámetros en Request objects para mayor flexibilidad y extensibilidad.
4. Limpieza de código técnico removiendo acentos innecesarios.

Todos los cambios mantienen la arquitectura de capas del proyecto (Dominio, Aplicación, Infraestructura, Consola) y siguen los principios SOLID, específicamente el Principio de Abierto/Cerrado que permite evolucionar el código sin romper clientes existentes.

El proyecto compila sin errores y está listo para su uso.